



APLICACION DE MODELOS AGILES EN EL DESARROLLO DE SOFTWARE

Kanban + Scrum + prácticas XP

David Martín Ceballos
Alejandro Braojos Manzano
Alexander Rodriguez Tapia
Roger A. Guevara Troche



INTRODUCCION

En el presente trabajo realizaremos un simulacro de proyecto aplicando las metodologías ágiles de desarrollo:

- Kanban
- Scrum
- XP (eXtreme Programming)

Enunciado:

SoftWareHouse quiere gestionar su negocio de ventas digitales de videojuegos mediante una app y exige que se lleven acabo metodologías ágiles para su desarrollo.

Sprint Goal

Objetivo de negocio y experiencia de usuario

Duración del Sprint: 2 Semanas

01

Consultar el catálogo con comodidad

Permitir que el cliente explore el catálogo haciendo scroll o utilizando filtros.

RESULTADO ESPERADO

02

Comprar directamente desde la app

Facilitar el proceso de compra y actualizar el inventario automáticamente.

RESULTADO ESPERADO

Sprint Backlog

Historias de usuario transformadas en tareas técnicas

Historia de usuario	Tarea asociada
Tener acceso al catálogo completo mediante la app	Diseñar esquema de Base de Datos
Poder filtrar juegos por género / plataforma	Programar consultas según opciones de filtrado
Llevar un registro automático de cada venta	Actualizar stock en tiempo real y registrar cada transacción

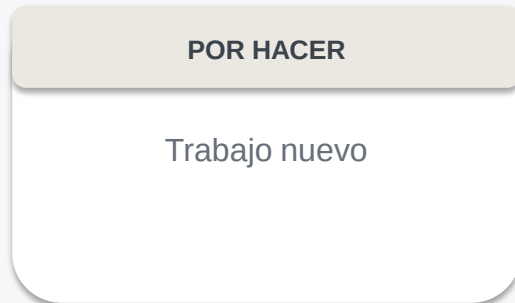
Método Kanban

Visualizar el trabajo para mejorar el flujo y la comunicación

1

Definir

Conjunto inicial de tareas



2

Visualizar

Columnas según el estado



3

Mover

Las tareas avanzan por etapas



Herramienta de referencia: KanbanFlow

Tablero Kanban

Evolución visual de las tareas durante el Sprint



Tareas principales del Sprint

Del diseño de datos a la compra y actualización del inventario

01 · Datos

- Diseñar la base de datos
- Crear tabla de videojuegos
- Crear tabla de ventas
- Crear tabla de usuarios/clientes
- Cargar videojuegos iniciales

02 · Catálogo

- Diseñar pantalla principal
- Mostrar catálogo
- Diseñar sistema de filtros
- Filtro por género
- Filtro por plataforma

03 · Compra

- Crear función de compra
- Actualizar stock después de una compra
- Registrar cada venta realizada

Daily Meeting

Seguimiento diario: ayer · hoy · impedimentos

Día 1

David

Ayer: comenzó los tests mínimos para actualizar el catálogo tras una venta.
Hoy: implementar la funcionalidad en un método.
Impedimento: ninguno.

Brajojos

Ayer: avanzó en la interfaz de filtrado.
Hoy: terminarla.
Impedimento: problemas de conexión al repositorio.

Día 2

David

Hoy: terminar el método de actualización.
Impedimento: dificultades por una API poco documentada.

Brajojos

Ayer: terminó la interfaz y resolvió el problema de red.
Hoy: presentación y estilo general del catálogo.
Impedimento: ninguno.

Prácticas XP aplicadas

Programación extrema para mejorar calidad, feedback y mantenibilidad



Programación en parejas

Dos desarrolladores trabajaron sobre el acceso al catálogo: uno escribe y otro revisa dinámicamente.

FEEDBACK CONTINUO



Test primero

Se definieron pruebas mínimas antes de implementar funcionalidades críticas.

CALIDAD DESDE EL INICIO



Refactorización

Se separaron responsabilidades para evitar lógica de descuentos y acceso a BD entrelazados.

CÓDIGO MÁS MANTENIBLE

Test mínimo: control del stock

Ejemplo de TDD aplicado a una operación crítica

`test_descontar_stock_tras_venta()`

Regla de negocio

El stock debe disminuir exactamente según las unidades vendidas.

5

stock inicial

- 2

3

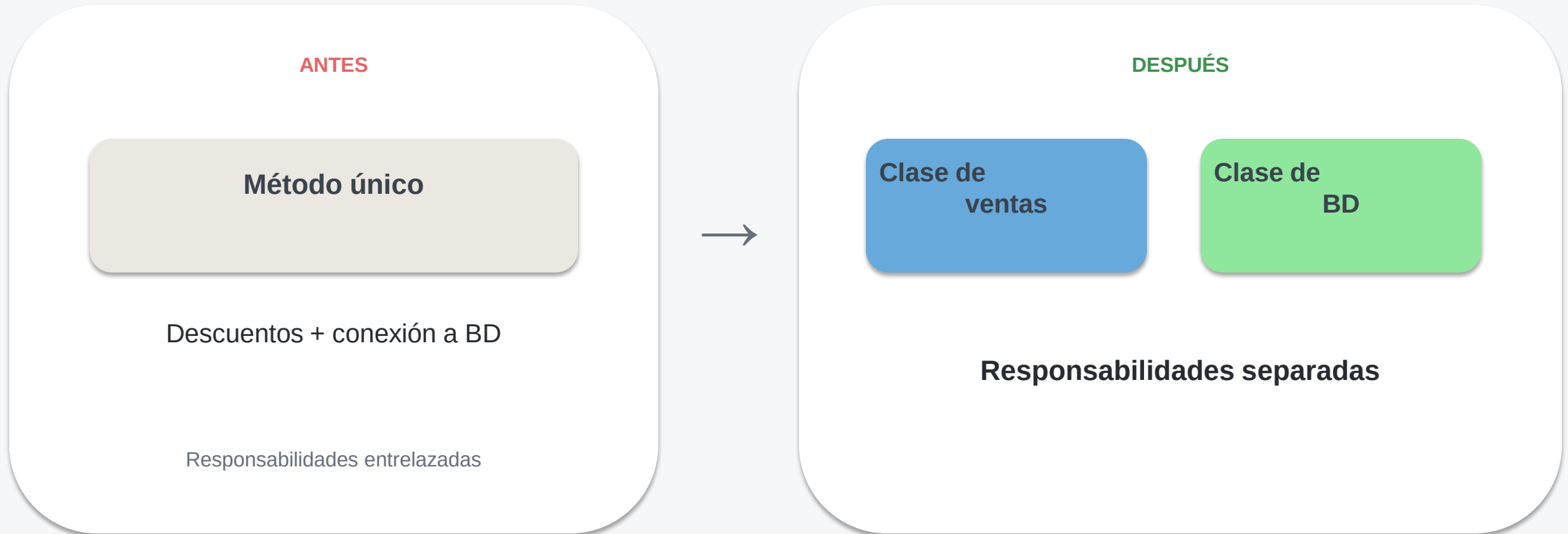
stock final

¿Qué valida el test?

- La venta se registra correctamente.
- El inventario se actualiza sin inconsistencias.
- El número de unidades restantes coincide con la operación.

Refactorización

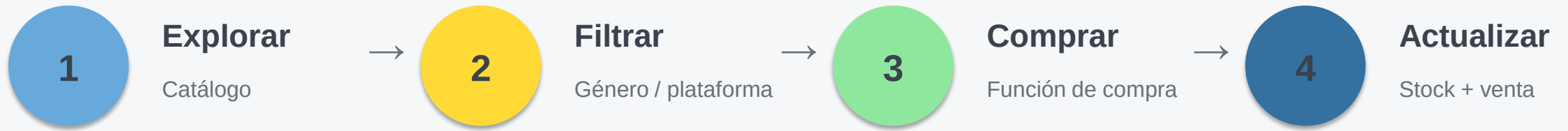
Separación de responsabilidades para reducir acoplamiento



→ **Menor acoplamiento · mayor mantenibilidad · cambios más seguros**

Flujo funcional del Sprint

Cómo se conecta la experiencia del usuario con el backend



Resultado técnico

Una compra desencadena la actualización del stock y el registro de la transacción, manteniendo consistente el catálogo.

Cierre del Sprint

De tareas aisladas a un flujo funcional

Kanban aportó visibilidad del trabajo y XP aportó calidad, feedback y capacidad de adaptación.

KANBAN

Flujo visible

XP

Calidad y feedback

SPRINT

Mínimo funcional

Gracias